

PSD lab 1

Uruchomienie i testowanie Apache Kafka

Krzysztof Fijałkowski

12.04.2026

1 PSD lab 1

1.1 Uruchomienie i testowanie Apache Kafka

Zgodnie z instrukcją, w pierwszym kroku utworzyłem service kafki.

```
kafka@ubuntu3:~/kafka$ sudo systemctl status kafka
* kafka.service
   Loaded: loaded (/etc/systemd/system/kafka.service; disabled; vendor preset: enabled)
   Active: active (running) since Thu 2026-04-09 14:45:47 CEST; 8s ago
     Main PID: 5439 (sh)
        Tasks: 72 (limit: 9437)
      Memory: 372.4M
         CPU: 13.836s
    CGroup: /system.slice/kafka.service
            └─5439 /bin/sh -c "/home/kafka/kafka/bin/kafka-server-start.sh $KAFKA_HOME/config/server.properties && java -Xmx1G -Xms1G -server -XX:+UseG1GC -XX:MaxGCPa
               `─5441 java -Xmx1G -Xms1G -server -XX:+UseG1GC -XX:MaxGCPa

kwi 09 14:45:47 ubuntu3 systemd[1]: Started kafka.service.
lines 1-12/12 (END)
```

Figure 1: stworzenie serwisu

Następnie wytworzyłem topic:

```
kafka@ubuntu3:~/kafka$ ~kafka/kafka/bin/kafka-topics.sh --create --bootstrap-server localhost:9092 --replication-factor 1 --partitions 1 --topic PSDTopic
Created topic PSDTopic.
```

Figure 2: stworzenie topicu

W celu testowania działania kafki ręcznie wysłałem wiadomość na testowym topic, a następnie odczytałem w konsoli za pomocą uruchomienia konsumenta.

Po uruchomieniu serwisu kafkadrop zobaczyłem swój topic, wchodząc w który mogłem sprawdzić, że wiadomość została poprawnie wysłana i odebrana,

```
kafka@ubuntu3:~/kafka$ echo "Hello, World" | ~kafka/kafka/bin/kafka-console-producer.sh --broker-list localhost:9092 --topic PSDTopic > /dev/null
kafka@ubuntu3:~/kafka$ ~kafka/kafka/bin/kafka-console-consumer.sh --bootstrap-server localhost:9092 --topic PSDTopic --from-beginning
Hello, World
```

Figure 3: stworzenie wiadomości

The screenshot shows the Kafdrop web interface running on localhost:9000. The interface displays the Kafka Cluster Overview with the following metrics:

Metric	Value
Bootstrap servers	localhost:9092
Total topics	2
Total partitions	51
Total preferred partition leader	100%
Total under-replicated partitions	0

Below the overview, the Brokers section shows a single broker:

ID	Host	Port	Rack	Controller	Number of partitions (% of total)
0	ubuntu3.myguest.virtualbox.org	9092	-	Yes	51 (100%)

The Topics section shows two topics:

Name	Partitions	% Preferred	# Under-replicated	Custom Config
PSDTopic	1	100%	0	No
__consumer_offsets	50	100%	0	Yes

A '+ New' button is visible at the bottom of the Topics section.

Figure 4: sprawdzenie topicu kafkadrop

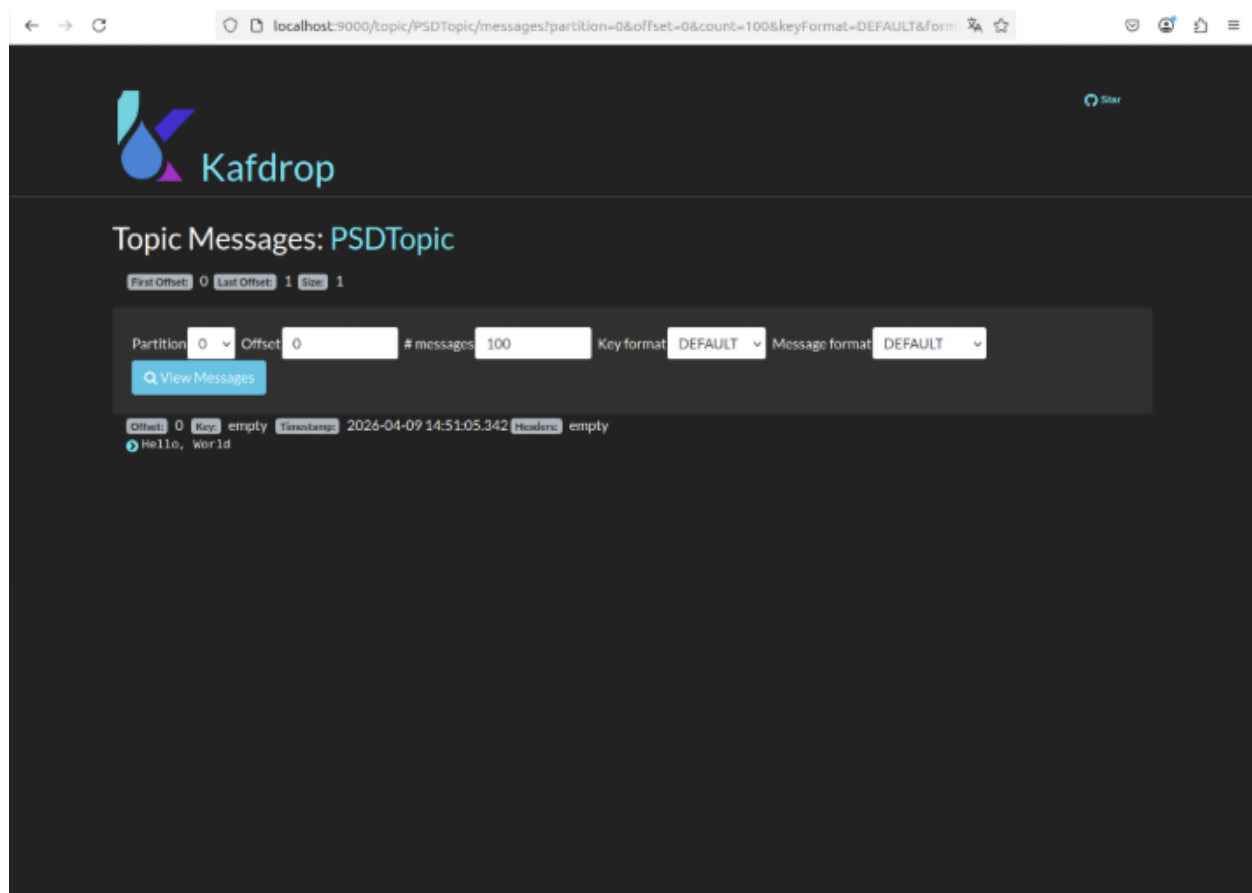


Figure 5: sprawdzenie wiadomości kafkadrop

1.2 Wytworzenie własnego producenta i konsumenta

W celu wytworzenia własnego producenta i konsumenta, skorzystałem z przykładowego kodu dostępnego w instrukcji. Zmodyfikowałem go tak, aby producent wysyłał wiadomość o aktualnej temperaturze w singapurze co sekundę, a konsument odbierał te wiadomości i wyświetlał je w konsoli.

Poniżej znajduje się kluczowy kod producenta:

```
def get_singapore_temperature():
    url = (
        "https://api.open-meteo.com/v1/forecast"
        "?latitude=1.3521&longitude=103.8198"
        "&current=temperature_2m"
    )
    response = requests.get(url, timeout=10)
    response.raise_for_status()
    data = response.json()

    return {
        "city": "Singapore",
        "temperature_c": data["current"]["temperature_2m"],
        "timestamp": datetime.now().isoformat()
    }

if __name__ == "__main__":
    while True:
        try:
            message = get_singapore_temperature()

            print(f'Producing message @ {datetime.now()} | Message = {message}')
            producer.send("messages", message)

        except Exception as e:
            print(f"Error fetching or sending temperature: {e}")

        time.sleep(1)
```

Konsumer pozostał bez znaczących zmian.